Pandas, Partie II

Exploration, Modification, et Tri

Slides adapted from Data 100/Data 200, Spring 2026 @ UC Berkeley

Base Election: questions analytiques possibles

Questions fréquentes

- Quel parti politique obtient le plus de victoires sur la période étudiée ?
- Quelle relation observe-t-on entre le pourcentage du vote populaire et le résultat final ?
- Existe-t-il des cas où un candidat gagne le vote populaire mais perd l'élection ?

Autres questions

- Identification de séries de victoires ou de défaites
- Évolution de la performance des partis au fil du temps
- Participation répétée de certains candidats
- Changements d'affiliation politique

Enrichissement possible de l'analyse

- Certaines questions nécessitent l'intégration de données externes :
- Impact des évolutions du droit de vote
- Influence du profil des candidats (ex. niveau d'éducation)
- Transformation et recomposition des partis politiques

Plan

- **Exploration d’un DataFrame**
- Ajout, suppression et modification des colonnes
- Tri des données

Récupérer `Index, Columns, & shape (Rappel)`

```
elections = pd.read_csv("data/elections.csv", index_col = "Candidate")
```

`DataFrame.index`: Pour extraire les *labels* de lignes

```
elections.index
```



```
Index(['Democratic-Republican', 'Democratic-Republican', 'Democratic',
       'National Republican', 'Democratic', 'National Republican',
       'Anti-Masonic', 'Whig', 'Democratic', 'Whig',
       ...
       'Green', 'Democratic', 'Republican', 'Libertarian', 'Green',
       'Republican', 'Democratic', 'Green', 'Independent',
       'Libertarian Party'],
      dtype='object', name='Party', length=187)
```

`DataFrame.columns`: Pour extraire les *labels* de colonnes

```
elections.columns
```

```
Index(['Candidate', 'Year', 'Popular vote', 'Result', '%'], dtype='object')
```

`DataFrame.shape`: Pour les dimensions

```
elections.shape
```

```
(187, 6)
```



`.shape` et `.size`

- L'attribut `.shape` fournit les dimensions sous forme d'un tuple **(# lignes, # colonnes)**.
- L'attribut `.size` fournit le nombre total d'éléments → # lignes multiplied by # colonnes.

`babynames`

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5
407424	CA	M	2022	Zia	5
407425	CA	M	2022	Zora	5
407426	CA	M	2022	Zuriel	5
407427	CA	M	2022	Zylo	5

407428 rows × 5 columns

```
babynames.shape
(407428, 5)
```



```
babynames.size
2037140
```



407,428 * 5 = 2,037,140

Les **Series** et **DataFrames** de Pandas prennent en charge de nombreuses opérations, y compris celles de **NumPy**, à condition que les données soient numériques.

```
yash_count = babynames.loc[babynames["Name"]=="Yash", "Count"]
```

```
np.mean(yash_count)
17.142857142857142
```

```
np.max(yash_count)
29
```



	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5
407424	CA	M	2022	Zia	5
407425	CA	M	2022	Zora	5
407426	CA	M	2022	Zuriel	5
407427	CA	M	2022	Zylo	5

```
331824     8
334114     9
336390    11
338773    12
341387    10
343571    14
345767    24
348230    29
350889    24
353445    29
356221    25
358978    27
361831    29
364905    24
367867    23
370945    18
374055    14
376756    18
379660    18
383338     9
385903    12
388529    17
391485    16
394906    10
397874     9
400171    15
403092    13
406006    13
Name: Count, dtype: int64
```

Les méthodes intégrées (*Built-In*) de `pandas`

`pandas` propose également un très grand nombre de fonctions utilitaires, notamment:

- `size/shape`
- `describe`
- `sample`
- `value_counts`
- `unique`
- `sort_values`
- …

`.describe()`

- La méthode `.describe()` renvoie des statistiques descriptives d'un `DataFrame` ou d'une `Series`.

`babynames`

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5
407424	CA	M	2022	Zia	5
407425	CA	M	2022	Zora	5
407426	CA	M	2022	Zuriel	5
407427	CA	M	2022	Zylo	5

407428 rows × 5 columns

`babynames.describe()`

	Year	Count
count	407428.000000	407428.000000
mean	1985.733609	79.543456
std	27.007660	293.698654
min	1910.000000	5.000000
25%	1969.000000	7.000000
50%	1992.000000	13.000000
75%	2008.000000	38.000000
max	2022.000000	8260.000000

Note: Only the numeric columns are summarized!

`.value_counts()`

La méthode `Series.value_counts()` renvoie le nombre d'occurrences de chaque valeur unique.

- Les valeurs retournées sont sous forme d'une `Series` triée.

```
babyname["Name"].value_counts()
```

```
Name
Jean          223
Francis       221
Guadalupe     218
Jessie        217
Marion        214
              ...
Ozair           1
Rahm            1
Ruhan           1
Sufiyan         1
Theon           1
Name: count, Length: 20437, dtype: int64
```



`.sample()`

Pour tirer aléatoirement des lignes d'un `DataFrame`, utilisez la méthode `.sample()`

- Par défaut, pandas effectue **un échantillonnage (*sampling*) sans remise**. Utilisez `replace=True` pour un `échantillonnage avec remise`.
  - Rappel : « échantillonnage avec remise » signifie que chaque ligne peut être tirée plus d'une fois.

```
babynames.sample()
```

	State	Sex	Year	Name	Count
121141	CA	F	1992	Shanelle	28

n=1 par défaut

✓*Le début d'un* `DataFrame` *n'est pas toujours représentatif, donc* `df.sample()` *est préférable à* `df.head()` *pour explorer les données.*

Enchaînement de méthodes

Les méthodes de `pandas` peuvent être enchaînées:

```
babynames.sample(5).iloc[:, 2:]
```

	Year	Name	Count
44448	1961	Karyn	36
260410	1948	Carol	7
397541	2019	Arya	11
4767	1921	Sumiko	16
104369	1987	Thomas	11



Le chaînage sur plusieurs lignes nécessite l'utilisation de parenthèses.

```
result = (
    babynames[babynames["Year"]==2000]
    .sample(4, replace=True)
    .iloc[:, 2:]
)
```

	Year	Name	Count
151749	2000	Iridian	7
343560	2000	Maverick	14
149491	2000	Stacy	91
149212	2000	Angel	307

.unique()

La méthode `Series.unique` renvoie un tableau contenant toutes les valeurs uniques d'une `Series`.

```
babynames["Name"].unique()
```



```
array(['Mary', 'Helen', 'Dorothy', ..., 'Zae', 'Zai', 'Zayvier'],
      dtype=object)
```

Plan

- Exploration d’un DataFrame
- **Ajout, suppression et modification des colonnes**
- Tri des données

Ajout d'une colonne: Syntaxe

Création d'une nouvelle colonne contenant le nombre de caractères de chaque prénom

1. `Series` avec la longueur de chaque prénom

```
babynames["name_lengths"] = babynames["Name"].str.len()
```

2. Affecter une Series à une nouvelle colonne appelée `"name_lengths"`.

	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	4
1	CA	F	1910	Helen	239	5
2	CA	F	1910	Dorothy	220	7
3	CA	F	1910	Margaret	163	8
4	CA	F	1910	Frances	134	7
...	...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5	7
407424	CA	M	2022	Zia	5	3
407425	CA	M	2022	Zora	5	4
407426	CA	M	2022	Zuriel	5	6
407427	CA	M	2022	Zylo	5	4

Modification d'une colonne: Syntaxe

Modifier la colonne `name_lengths` pour que chaque valeur soit égale à sa valeur initiale moins 1

```
babynames["name_lengths"] = babynames["name_lengths"] - 1
```

```
babynames.loc[:, "name_lengths"] = babynames["name_lengths"] - 1
```

	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6
...	...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5	6
407424	CA	M	2022	Zia	5	2
407425	CA	M	2022	Zora	5	3
407426	CA	M	2022	Zuriel	5	5
407427	CA	M	2022	Zylo	5	3



	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	4
1	CA	F	1910	Helen	239	5
2	CA	F	1910	Dorothy	220	7
3	CA	F	1910	Margaret	163	8
4	CA	F	1910	Frances	134	7
...	...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5	7
407424	CA	M	2022	Zia	5	3
407425	CA	M	2022	Zora	5	4
407426	CA	M	2022	Zuriel	5	6
407427	CA	M	2022	Zylo	5	4

- Pour modifier certaines lignes d'une colonne, utilisez `.loc` pour cibler précisément les lignes à changer.

Renommer une colonne

Renommer une colonne en utilisant la méthode `.rename()`.

```
# Rename "name_lengths" to "Length"
babynames = babynames.rename(columns={"name_lengths":"Length"})
```



	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6
...	...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5	6
407424	CA	M	2022	Zia	5	2
407425	CA	M	2022	Zora	5	3
407426	CA	M	2022	Zuriel	5	5
407427	CA	M	2022	Zylo	5	3



	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6
...	...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5	6
407424	CA	M	2022	Zia	5	2
407425	CA	M	2022	Zora	5	3
407426	CA	M	2022	Zuriel	5	5
407427	CA	M	2022	Zylo	5	3

Supprimer une colonne (ou une ligne): Syntaxe

Supprimer une colonne en utilisant la méthode `.drop`.

- La méthode .drop() supprime par défaut des lignes. Pour supprimer des colonnes, utilisez axis=`"columns"`.



```
babynames = babynames.drop("Length", axis="columns")
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5
407424	CA	M	2022	Zia	5
407425	CA	M	2022	Zora	5
407426	CA	M	2022	Zuriel	5
407427	CA	M	2022	Zylo	5



	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6
...	...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5	6
407424	CA	M	2022	Zia	5	2
407425	CA	M	2022	Zora	5	3
407426	CA	M	2022	Zuriel	5	5
407427	CA	M	2022	Zylo	5	3

An Important Note: `DataFrame` Copies

Dans le slide précédent, nous avons réaffecté babynames à un DataFrame mis à jour.

```
babynames = babynames.drop("Length", axis="columns")
```

Par défaut, `pandas` crée une nouvelle **copie** du `DataFrame`, **sans modifier** le `DataFrame` original.

```
babynames.drop("Length", axis="columns")
babynames
```



	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6
...	...	...	...	...	...	...

Our change was not applied!

Supprimer une colonne (ou une ligne): Syntaxe (2)

`df.drop(1)`	Supprime la ligne d'index `1`.
`df.drop([0, 2])`	Supprime les lignes d'index `0` et `2`.
`df.drop("Age", axis=1)`	Supprime la colonne `Age`.
`df.drop(columns=["Age"])`	Supprime la colonne `Age`, sans préciser `axis`.
`df.drop(index=[0, 2])`	Supprime les lignes d'index `0` et `2`.

Plan

- Exploration d’un DataFrame
- Ajout, suppression et modification des colonnes
- **Tri des données**

`.sort_values()`

La méthode `.sort_values()` trie un `DataFrame` ou une `Series`.

- `DataFrame.sort_values(column_name)` → Pour un DataFrame, il faut spécifier la colonne à trier!

```
babynames.sort_values(by="Count", ascending=False)
```

	State	Sex	Year	Name	Count
268041	CA	M	1957	Michael	8260
267017	CA	M	1956	Michael	8258
317387	CA	M	1990	Michael	8246
281850	CA	M	1969	Michael	8245
283146	CA	M	1970	Michael	8196
...	...	...	...	...	...
317292	CA	M	1989	Olegario	5
317291	CA	M	1989	Norbert	5
317290	CA	M	1989	Niles	5
317289	CA	M	1989	Nikola	5
407427	CA	M	2022	Zylo	5

407428 rows × 5 columns

Par défaut, les lignes sont triées par ordre croissant.

Le tri réorganise les lignes, et l'index se déplace avec elles

```
babynames["Name"].sort_values()
```

```
366001      Aadan
384005      Aadan
369120      Aadan
398211    Aadarsh
370306      Aaden
           ...
220691      Zyrah
197529      Zyrah
217429      Zyrah
232167      Zyrah
404544      Zyrus
Name: Name, Length: 407428, dtype: object
```



Trier selon une valeur calculée

Supposons que l’on souhaite trier les données en fonction de la longueur de chaque prénom.

Approche classique :

- Créer une nouvelle colonne contenant la longueur de chaque prénom.
- Trier le DataFrame en utilisant cette nouvelle colonne.

Approche 1 : créer une nouvelle colonne et trier à partir de celle-ci

```
babynames["name_lengths"] = babynames["Name"].str.len()

babynames = babynames.sort_values(by="name_lengths", ascending=False)

babynames.head(5)
```

	State	Sex	Year	Name	Count	name_lengths
334166	CA	M	1996	Franciscojavier	8	15
337301	CA	M	1997	Franciscojavier	5	15
339472	CA	M	1998	Franciscojavier	6	15
321792	CA	M	1991	Ryanchristopher	7	15
327358	CA	M	1993	Johnchristopher	5	15

Approche 2 : trier à l'aide de l'argument key

```
babynames.sort_values("Name", key=lambda x: x.str.len(), ascending=False).head()
```

Créer une colonne temporaire contenant la longueur de chaque prénom, trier selon cette colonne, puis la supprimer.

	State	Sex	Year	Name	Count
334166	CA	M	1996	Franciscojavier	8
327472	CA	M	1993	Ryanchristopher	5
337301	CA	M	1997	Franciscojavier	5
337477	CA	M	1997	Ryanchristopher	5
312543	CA	M	1987	Franciscojavier	5

Rappel: `lambda x: x.str.len()` est une **fonction anonyme** (c'est-à-dire sans nom, temporaire et utilisée une seule fois). Cette fonction anonyme est appliquée à la ou aux colonnes utilisées pour le tri.

Approche 3 : trier à l'aide de la méthode `.map`

Tri basé sur le nombre d'occurrences d'un caractère ou d'une sous-chaîne, par exemple `'dr' and 'ea'`, avec la méthode `Series.map`

```
# Returns number of times 'dr' and 'ea' appear in `string`
def dr_ea_count(string):
    return string.count('dr') + string.count('ea')

# Apply dr_ea_count to each name in the "Name" column
babynames["dr_ea_count"] = babynames["Name"].map(dr_ea_count)


babynames.sort_values(by="dr_ea_count", ascending=False).head()
```

	State	Sex	Year	Name	Count	dr_ea_count
115957	CA	F	1990	Deandrea	5	3
101976	CA	F	1986	Deandrea	6	3
131029	CA	F	1994	Leandrea	5	3
108731	CA	F	1988	Deandrea	5	3
308131	CA	M	1985	Deandrea	6	3

Autres Methodes de transformations

0. Opérations vectorisées — transformer des colonnes entières

Privilégiez, lorsque c'est possible, les opérations arithmétiques et les comparaisons intégrées de Series/DataFrame plutôt que les boucles Python.

EXEMPLE

```
df = pd.DataFrame({
    "price": [10, 20, 15],
    "quantity": [2, 3, 4]
})

df["total"] = df["price"] * df["quantity"]
df["discounted"] = df["total"] * 0.90
df["large_order"] = df["total"] >= 50
```

RÉSULTAT

price	quantity	total	discounted	large_order
10	2	20	18.0	False
20	3	60	54.0	True
15	4	60	54.0	True

1. apply() — logique personnalisée sur les lignes

Utilisez DataFrame.apply(axis=1) lorsqu'un calcul dépend de plusieurs colonnes.

EXEMPLE

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Omar"],
    "salary": [5000, 4000, 6000],
    "experience": [3, 5, 7]
})

df["bonus"] = df.apply(
    lambda r: r["salary"] * 0.10
    if r["experience"] >= 5
    else r["salary"] * 0.05,
    axis=1
)
```

RÉSULTAT

name	salary	experience	bonus
Ali	5000	3	250
Sara	4000	5	400
Omar	6000	7	600

KEY IDEA axis=1 passes each row to the function; axis=0 applies to each column.

Conseil : pour les opérations arithmétiques et conditions simples, les opérations vectorisées sont généralement plus rapides que apply() ligne par ligne.

```
df["bonus"]=df[["salary"]].apply(lambda r: r * 0.10, axis=0)
```

2. map() — convertir les valeurs d'une Series

Utilisez Series.map() pour rechercher des valeurs dans un dictionnaire ou transformer les valeurs une par une.

EXEMPLE

```
df = pd.DataFrame({
    "name": ["Ali", "Sara", "Omar"],
    "grade": ["A", "B", "A"]
})

grade_points = {
    "A": 90, "B": 80, "C": 70
}

df["points"] = df["grade"].map(grade_points)

# Les clés sans correspondance deviennent NaN
```

RÉSULTAT

name	grade	points
Ali	A	90
Sara	B	80
Omar	A	90

KEY IDEA **map() transforms each Series value independently; it does not combine columns.**

API associée : DataFrame.map() applique une fonction à chaque cellule dans les versions récentes de pandas (pandas 2.1+).

```
df["bonus"] = df["salary"].map(lambda x: x * 0.10)
```

3. query() — sélectionner des lignes avec des conditions

Utilisez query() pour filtrer les lignes à l'aide d'une expression sous forme de chaîne de caractères.

EXEMPLE

```
df = pd.DataFrame({
    "name": ["Ali", "Sara", "Omar"],
    "salary": [5000, 4000, 6000],
    "experience": [3, 5, 7]
})

result = df.query("salary >= 5000 and experience >= 5")

# Équivalent avec un masque booléen :
result = df[(df["salary"] >= 5000) &
            (df["experience"] >= 5)]
```

RÉSULTAT

name	salary	experience
Omar	6000	7

KEY IDEA query() selects rows; it does not transform column values.

Pour les variables Python externes, utilisez @variable dans l'expression query.

4. transform() — renvoyer des résultats alignés sur l’entrée

Series/DataFrame.transform applique une transformation tout en préservant la forme de l’entrée pour les opérations courantes.

EXEMPLE

```
df = pd.DataFrame({
    "salary": [5000, 4000, 6000]
})

df["salary_z"] = df["salary"].transform(
    lambda s: (s - s.mean()) / s.std()
)

# Dans cet exemple, pandas utilise l’écart-type
d’échantillon (ddof=1).
```

RÉSULTAT

index	salary	salary_z (approx.)
0	5000	0.218
1	4000	-0.873
2	6000	0.655

KEY IDEA transform() is useful when the result must align with the original rows.

Contrairement à agg(), transform() est conçu pour renvoyer un résultat de même forme ou pouvant être diffusé (broadcast).

map() vs apply() vs transform()

Même résultat dans les cas simples — entrées de fonction et usages prévus différents

1 MÊME TÂCHE : DOUBLER CHAQUE SALAIRE

```
s = pd.Series([1000, 2000, 3000])

s.map(lambda x: x * 2)
s.apply(lambda x: x * 2)
s.transform(lambda x: x * 2)
```

Ici, les trois méthodes renvoient les mêmes valeurs :

2000 **4000** **6000**

2 UN CAS OÙ ELLES DIFFÈRENT

```
s = pd.Series([1000, 2000, 3000])

s.transform(lambda x: x - x.mean())
# [-1000, 0, 1000]

s.map(lambda x: x - x.mean())  # Error
```

Pourquoi ?
transform() transmet la Series entière à la fonction ;
map() transmet une valeur scalaire à la fois.

map()

Une valeur à la fois

Idéal pour les recherches dans un dictionnaire et les correspondances valeur à valeur.

apply()

Fonction flexible

Series : généralement valeur par valeur. DataFrame : peut traiter les lignes ou les colonnes.

transform()

Préserve la forme

Conçu pour renvoyer des résultats alignés sur la Series/DataFrame d'origine.

RÈGLE PRATIQUE : map = associer des valeurs • apply = logique flexible • transform = transformation alignée

assign()

1227261

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Ali", "Sara", "Omar"],
    "salary": [3000, 4000, 5000],
    "experience": [2, 5, 8]
})

result = df.assign(
    annual_salary=df["salary"] * 12
)

print(result)
```

	name	salary	experience	annual_salary
0	Ali	3000	2	36000
1	Sara	4000	5	48000
2	Omar	5000	8	60000

assign() : crée ou met à jour des colonnes du DataFrame

6. pipe() — composer des étapes de transformation réutilisables

Utilisez pipe() pour transmettre un DataFrame ou une Series entier à une fonction nommée.

EXEMPLE

```
def add_annual_salary(data):
    return data.assign(
        annual_salary=data["salary"] * 12
    )

result = (
    df.pipe(add_annual_salary)
      .query("experience >= 5")
)

# On suppose que df contient les colonnes salary
et experience.
```

RÉSULTAT

name	salary	experience	annual_salary
Sara	4000	5	48000
Omar	6000	7	72000

KEY IDEA pipe() improves readability by letting reusable functions fit into a method chain.

pipe() ne définit pas lui-même une transformation ; il transmet l'objet à votre fonction.

add_annual_salary(df) == df.pipe(add_annual_salary)

1227261

Sans .pipe() :

```
df1 = add_annual_salary(df)
df2 = df1.query("experience >= 5")
df3 = df2.sort_values("annual_salary")
```

```
df3 = add_annual_salary(df).query("experience >=
5").sort_values("annual_salary")
```

Avec .pipe() :

```
result = (
df.pipe(add_annual_salary)
.query("experience >= 5")
.sort_values("annual_salary")
)
```

7. where() — conserver les valeurs qui satisfont une condition

Les valeurs pour lesquelles la condition est False sont remplacées par la valeur de remplacement choisie.

EXEMPLE

```
df = pd.DataFrame({
    "name": ["Ali", "Sara", "Omar"],
    "salary": [5000, 4000, 6000]
})

df["eligible_salary"] = df["salary"].where(
    df["salary"] >= 5000,
    other=0
)
```

RÉSULTAT

name	salary	eligible_salary
Ali	5000	5000
Sara	4000	0
Omar	6000	6000

KEY IDEA **where(condition, other) keeps values where condition is True.**

Sans other=..., les valeurs qui ne satisfont pas la condition sont généralement remplacées par une valeur manquante.

8. mask() — remplacer les valeurs qui satisfont une condition

mask() est le complément inversé de where() : il remplace les valeurs pour lesquelles la condition est True.

EXEMPLE

```
df = pd.DataFrame({
    "name": ["Ali", "Sara", "Omar"],
    "salary": [5000, 4000, 6000]
})

df["capped_salary"] = df["salary"].mask(
    df["salary"] > 5000,
    other=5000
)
```

RÉSULTAT

name	salary	capped_salary
Ali	5000	5000
Sara	4000	4000
Omar	6000	5000

KEY IDEA mask(condition, other) replaces values where condition is True.

Astuce mémoire : where conserve les True ; mask remplace les True.